Master en Desarrollo Full Stack · ConquerBlocks

Guia para defender ALPRO TECHNOLOGY SPA

Documento de estudio. Explica el problema, los datos, la arquitectura, los flujos y las decisiones tecnicas del proyecto, con el lenguaje que conviene usar en la videollamada.

Repositorio: https://github.com/yondaimejin4/tareasConquerblocks
Carpeta: proyecto final master full stack
Aplicacion local: http://127.0.0.1:8000/
Admin de prueba: usuario admin · contrasena Admin123

Antes de la defensa, ten abiertos: la URL publica, este proyecto en el editor con el servidor local en marcha, y el repositorio en GitHub.

1. Como explicar el proyecto en un minuto

ALPRO TECHNOLOGY SPA es la web de una empresa de servicios tecnologicos. El problema es que las consultas comerciales llegan por WhatsApp, correo y llamadas, sin un catalogo unico ni un seguimiento del proyecto. La solucion es una aplicacion con tres superficies: una landing publica en React, un portal para el cliente y el panel de Django Admin para el equipo.

El visitante ve servicios que salen de la base de datos y puede pedir una consultoria. El cliente entra con usuario y contrasena, crea solicitudes y aporta ideas o imagenes. El administrador cambia el estado del proyecto, el estado de pago y deja un comentario que el cliente ve al recargar. No hay pasarela de pago: el pago se registra a mano, que es el alcance del MVP.

2. Que se ve en cada pantalla

Landing publica /

- Catalogo completo y servicios destacados, cargados con fetch a la API.
- Formulario Solicita una consultoria: nombre, correo, empresa y mensaje.
- Boton de WhatsApp y enlace al area de cliente.

Portal /portal/login

- Registro e inicio de sesion. Al entrar aparece Bienvenido y el nombre de usuario.
- Mis solicitudes: lista, alta y detalle.
- En el detalle: estado del proyecto, comentario del equipo, estado de pago y aportes.

Admin /admin/

- Usuarios: crear cliente con correo y contrasena, y ver sus solicitudes.
- Solicitudes de clientes: mensaje, aportes, estado, comentario y pago.
- Servicios: alta, edicion y marca de destacado.

Plantilla Django /django/

Es una vista servidor adicional, pedida como parte del backend con plantillas. La experiencia

principal del visitante es la SPA de React, no esta plantilla.

3. Datos: que se guarda y quien lo ve

Hay cuatro conjuntos de datos. User es el usuario de Django. Service es el catalogo. ContactLead es la solicitud o lead. ClientContribution es un aporte del cliente ligado a una solicitud.

Service

title, short_description, details, is_featured y created_at. El visitante solo lee. El administrador crea y edita. seed_demo carga 16 servicios, 5 de ellos destacados, desde backend/core/fixtures/initial_data.json.

ContactLead

Puede nacer del formulario publico o del portal. Guarda user (opcional), name, email, company, message, status, payment_status, pending_summary, team_comment, admin_notes, created_at y updated_at.

- status: recibida, en revision, en desarrollo, entregada.
- payment_status: pendiente, pago parcial, pagado.
- team_comment y pending_summary los escribe el admin y los lee el cliente.
- admin_notes es interno. No sale en la API del portal.

ClientContribution

lead, user, message, attachment (imagen opcional en media/contributions/) y created_at. Si se borra la solicitud, se borran sus aportes (CASCADE).

Relaciones

Un usuario tiene muchas solicitudes (project_requests). Una solicitud tiene muchos aportes (contributions). Un aporte pertenece a un usuario y a una solicitud. El catalogo no depende de los usuarios.

Si el formulario publico se envia sin sesion de portal, user queda vacio: es un lead anonimo. Si el cliente esta logueado en el portal y crea la solicitud desde Mis solicitudes, user es ese cliente.

4. Arquitectura

El navegador habla con un solo origen. En local es http://127.0.0.1:8000. Django sirve la API REST y, despues de npm run build, tambien el HTML y los archivos de React que estan en frontend/dist. Por eso no hace falta CORS en el modo recomendado: la pagina y la API comparten dominio.

Cadena: React (Vite) -> JSON por HTTP -> Django y Django REST Framework -> SQLite en desarrollo o PostgreSQL en produccion (DATABASE_URL). WhiteNoise sirve los estaticos del admin. Gunicorn es el servidor de produccion.

Carpetas que debes saber senalar

- backend/core/models.py: entidades.
- backend/core/views.py y urls.py: API.

- backend/core/serializers.py: validacion de entrada y forma del JSON.
- backend/core/admin.py y admin_users.py: panel.
- backend/core/portal_auth.py y authentication.py: sesion del cliente.
- frontend/src/pages/LandingPage.jsx y Portal.jsx: pantallas React.
- frontend/src/api/client.js: fetch, cookies y CSRF.
- scripts/dev.sh: migrar, sembrar datos, compilar y arrancar.

5. Como React obtiene datos reales

Esto es el punto que la guia pide demostrar. En LandingPage, al montar el componente, fetchServices() llama a GET /api/services/ y a GET /api/services/featured/. La respuesta es JSON generado por ServiceSerializer a partir de la tabla Service. React pinta tarjetas. Si el backend esta apagado, la pantalla muestra error en lugar de inventar datos.

El formulario de consultoria hace POST /api/contact/ con nombre, correo, empresa y mensaje. ContactLeadCreateSerializer rechaza un correo invalido con HTTP 400. Si es valido, se crea la fila y React muestra el mensaje de gracias.

En el portal, las listas no usan la sesion del admin. Usan la cookie portal_sessionid. GET /api/my-requests/ devuelve solo las solicitudes cuyo user es el cliente autenticado.

6. Dos sesiones a proposito

Django Admin y el portal viven en el mismo dominio. Si compartieran la cookie sessionid, un administrador con el panel abierto crearia solicitudes como admin al usar el portal en otra pestana. Por eso el portal no usa login() de Django sobre la sesion normal.

portal_login guarda el id del usuario en una sesion propia y la entrega en la cookie portal_sessionid. PortalSessionAuthentication solo lee esa cookie. La API de contacto publico tiene authentication_classes vacio, para que la cookie del admin no dispare un error CSRF ni asigne el lead al admin.

Frase para la defensa: separe las sesiones porque el administrador y el cliente pueden usar el mismo navegador, y cada solicitud debe quedar a nombre de quien realmente la envia.

7. Recorrido que conviene ensenar en vivo

- Arranque: ./scripts/dev.sh desde la carpeta del proyecto.
- Landing: senalar en la pestana Red las peticiones /api/services/.
- Enviar una consultoria y abrirla en Admin, Solicitudes de clientes.
- Crear un usuario cliente en Admin y entrar en /portal/login.
- Crear una solicitud y un aporte. Volver al admin y ensenar el inline.
- Cambiar estado, comentario y pago. Recargar el detalle del cliente.
- Opcional: python manage.py test core. Hay pruebas de API, aislamiento entre clientes, sesion de admin que no contamina el portal, y comentario visible.

8. Seguridad, en palabras simples

- Contrasenas: Django las guarda con PBKDF2. En el codigo no hay claves en claro.

- Validacion: los serializers no aceptan campos de mas ni correos rotos.
- CSRF: el portal pide /api/auth/csrf/ y envia X-CSRFToken en POST.
- XSS: React escapa el texto. Las plantillas Django tambien.
- Permisos: un cliente que pide el id de otra solicitud recibe 404.
- Secretos: SECRET_KEY, DEBUG y ALLOWED_HOSTS salen de variables de entorno. .env no se sube.
- HTTPS y DEBUG=False: obligatorios en el despliegue publico, no en el portatil.
- Copias: en produccion la base es PostgreSQL del proveedor; el backup es responsabilidad de ese servicio.

Lo que no hace esta version, y hay que decirlo si preguntan: no cifra datos en reposo mas alla de lo que ofrezca el hosting, no tiene doble factor, y no cobra con tarjeta. El pago es un estado manual.

9. Preguntas probables y una respuesta corta

Por que Django y React, y no solo plantillas?

Django cubre modelos, admin, autenticacion y persistencia. React cubre la landing y el portal consumiendo la API, que es el requisito de integracion del master. La plantilla /django/ demuestra que el backend tambien renderiza HTML.

Por que la misma origen y no dos servidores?

En desarrollo, dos puertos provocaban fallos de CORS y de cookies. Compilar React y servirlo desde Django deja una sola URL y unas cookies fiables.

Donde se valida el formulario?

En el navegador hay required y type=email. La validacion que cuenta esta en el serializer del servidor. El cliente no es de confianza.

Que pasa si borro un usuario?

La solicitud no se borra: user pasa a null (SET_NULL). Los aportes del usuario si se borran, porque su clave foranea es CASCADE.

Como se crean los datos de demo?

python manage.py seed_demo lee el fixture. seed_admin crea o actualiza el superusuario admin con contrasena Admin123. En una entrega real hay que decir esas credenciales en el comentario de Academia y cambiarlas si el sitio queda publico mucho tiempo.

10. Lo que la guia v2 todavia exige fuera de este PDF

Este archivo prepara la defensa. No sustituye los cuatro entregables. Segun la guia de octubre de 2026 faltan, si aun no se han hecho:

- URL publica HTTPS con DEBUG desactivado, datos de ejemplo y un usuario de prueba por rol.
- Memoria PDF de la academia: portada con nombre, correo, convocatoria y enlaces; capturas; diagrama; declaracion de uso de IA; limitaciones y conclusiones. Entre 20 y 40 paginas, texto

seleccionable.

- Video de menos de 5 minutos: problema, demo, tecnologias y React leyendo la API. Enlace que abra sin pedir permiso.
- En Academia: URL del repo, URL de la app, PDF, video y credenciales en el comentario.

Credenciales de demo para ese comentario: administrador admin / Admin123. El cliente de prueba es el que se cree en Admin, Usuarios, Anadir usuario.

Repo ya publicado: https://github.com/yondaimejin4/tareasConquerblocks dentro de la carpeta proyecto final master full stack.

11. Guion breve para los 30 minutos

- Minuto 0 a 3: problema, usuarios y que automatiza la herramienta.
- Minuto 3 a 12: demo en la URL publica. Landing, formulario, portal, admin.
- Minuto 12 a 18: ensenar models.py, la vista de servicios y el fetch de LandingPage.
- Minuto 18 a 24: explicar las dos cookies y un test que lo demuestra.
- Minuto 24 a 30: seguridad, limites del MVP y que harias despues (correo automatico, pago real, backup).

Si piden un cambio pequeno en directo, lo mas seguro es un texto de la landing, un choice de estado o un campo del serializer. No rehagas la autenticacion en la llamada.